

Project Report

Tic-Tac-Toe Game Using Python (Tkinter)

Submitted by

Rachita Joshi

Registration Number: 26BAI10627

1. Introduction

Tic-Tac-Toe is a classic two-player paper-and-pencil game played on a 3x3 grid. This project implements a graphical version of the game in Python using the built-in Tkinter library. The application allows two human players to take turns placing their marks (X or O) on the board, automatically detects a win or a draw, and displays the result through a pop-up message.

2. Objective

The primary objective of this project is to build a simple, interactive, and fully functional Tic-Tac-Toe game with a graphical user interface (GUI), while reinforcing core programming concepts such as:

- Event-driven programming using GUI callbacks
- Working with global state and game logic
- Implementing win/draw detection algorithms
- Designing a clean and usable user interface

3. Tools and Technologies Used

Component	Details
Programming Language	Python 3.x
GUI Library	Tkinter (standard library)
Dialogs	tkinter.messagebox
IDE/Environment	Any Python-compatible editor (VS Code, PyCharm, IDLE, etc.)

4. System Design and Methodology

The application follows a simple event-driven design. The Tkinter root window hosts a 3x3 grid of buttons representing the game board, along with a label that displays whose turn it currently is. Each button click is handled by a callback function that updates the game state and checks for a win or draw condition after every move.

4.1 Game Flow

1. The board is initialized with nine empty buttons and Player X begins the game.
2. When a player clicks an empty cell, their symbol (X or O) is placed on it.

3. The program checks all eight possible winning combinations (3 rows, 3 columns, 2 diagonals).
4. If a winning combination is found, the matching cells are highlighted in green and a pop-up announces the winner.
5. If no winner is found and all cells are filled, a pop-up announces a tie.
6. If the game is not yet decided, control passes to the other player and the turn label updates.

4.2 Core Functions

Function	Purpose
check_winner()	Scans all 8 winning combinations; highlights and announces a win if found
check_draw()	Checks if the board is full with no winner and announces a tie
button_click(index)	Handles a cell click: places the mark, checks win/draw, toggles turn
toggle_player()	Switches the active player and updates the turn label

5. Features

- 3x3 interactive game board built with Tkinter buttons
- Real-time turn indicator label
- Automatic detection of all winning combinations
- Visual highlight (green) of the winning line
- Draw detection when the board is full
- Pop-up dialogs for win/tie announcements

6. Sample Code Snippet

The win-checking logic lies at the heart of the game and is implemented as follows:

```
win_combos = [  
    [0,1,2],[3,4,5],[6,7,8],  
    [0,3,6],[1,4,7],[2,5,8],  
    [0,4,8],[2,4,6]  
]  
for combo in win_combos:  
    if buttons[combo[0]]["text"] == buttons[combo[1]]["text"]  
       == buttons[combo[2]]["text"] != "":  
        # highlight and declare winner
```

7. Known Issues and Limitations

- The Tkinter root window (tk.Tk()) is instantiated twice in the current script, resulting in an extra, unused window being created.
- The list of board buttons is built twice, once before and once after the main setup block, leading to duplicate widgets being created (only the second set is actually displayed and used).
- There is no in-app "Restart" or "New Game" button; the script must be re-run to play again.
- The game is designed for two human players only; there is no computer/AI opponent.

8. Future Enhancements

- Remove the duplicate window/button initialization for cleaner code
- Add a 'Restart' or 'New Game' button to reset the board without closing the app
- Implement a single-player mode with a computer opponent (e.g., Minimax algorithm)
- Add a scoreboard to track wins/losses/draws across multiple rounds
- Improve the visual design with custom themes, colors, and fonts

9. Conclusion

This project successfully demonstrates the implementation of a classic Tic-Tac-Toe game using Python's Tkinter library. It covers essential GUI programming concepts such as event handling, dynamic widget updates, and game-state management. With minor code cleanup and the addition of features like a restart button and an AI opponent, the project can be extended into a more polished and feature-rich application.